

SONIC

Sensor-based Optimization for Navigation, Intelligence and Control

Optimisation par capteurs pour la navigation, l'intelligence et le contrôle

Course autonome à grande vitesse

Projet D – Robotique mobile autonome

HORDOIR Stéphane LATOUNDJI Mourad ZHANG Chengbo

Encadrant : YAN Zhi

Résumé exécutif

Le projet développe une pile de navigation autonome haute vitesse pour un véhicule Ackermann simulé sous ROS 2 Humble et Gazebo 11. Trois contrôleurs sont comparés ; Pure Pursuit est retenu pour son meilleur compromis vitesse/stabilité. Sur le circuit F1 mis à l'échelle, la combinaison *racing line* + lookahead adaptatif réduit le temps médian de 174,6 s à 86,7 s sans sortie de piste. Une pile plus autonome, Pure Pursuit + LiDAR + SLAM, boucle le circuit sans carte préalable en 115,1 s, mais montre une forte sensibilité à la latence et au bruit odométrique.

Dépôt GitHub

Code source, configurations et scripts d'expérimentation : github.com/mlatoundji/High-speed-autonomous-racing-SONIC-ROS2/tree/main.

Table des matières

1	Présentation de l'équipe et objectifs	1
2	Architecture du système	2
3	Méthodologie et implémentation	3
3.1	Sélection et réglage du contrôleur	3
3.2	Racing line et profil de vitesse	3
3.3	Pile Pure Pursuit + LiDAR + SLAM	4
4	Résultats expérimentaux	4
4.1	Protocole	4
4.2	Stage I : sélection du contrôleur sur circuit en anneau	5
4.3	Stage II : F1 open-loop, ligne centrale, racing line et tuning	5
4.4	Stage III : pile Pure Pursuit + LiDAR + SLAM	5
5	Robustesse : latence et bruit odométrique	6
6	Discussion : conservateur vs agressif	6
7	Limites et validité des essais	7
8	Leçons apprises	7
9	Conclusion et perspectives	7

1 Présentation de l'équipe et objectifs

Le travail a été réparti par couches logicielles, avec des responsabilités claires et des tâches transverses partagées pour l'intégration, les tests et la rédaction. Cette organisation a permis de séparer les enjeux de simulation, localisation, planification et contrôle, tout en conservant une boucle d'intégration courte lors des campagnes expérimentales.

Table 1 – Répartition des tâches par membre.

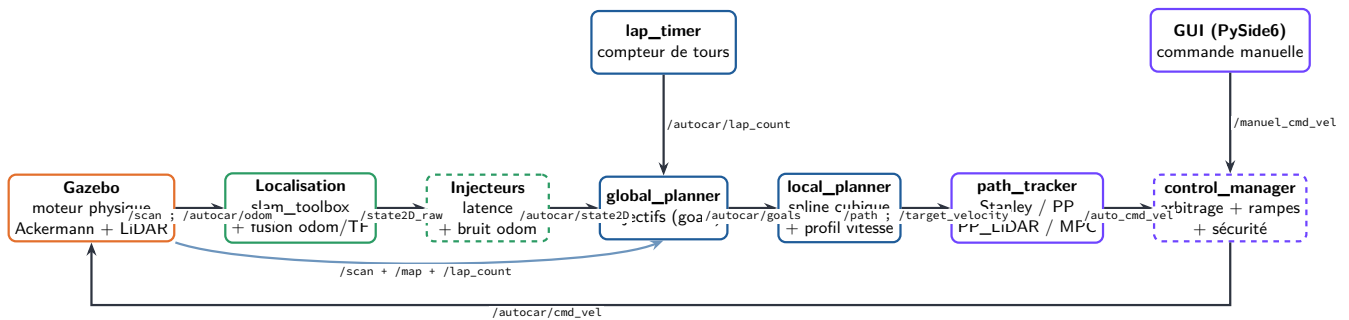
Membre	Responsabilité	Contribution détaillée
HORDOIR Stéphane	Planification et trajectoire	Modèle Ackermann sous Gazebo, capteur LiDAR, génération de la <i>racine line</i> par optimisation à courbure minimale, lissage et intégration <code>global_planner/local_planner</code> .
LATOUNDJJI Mourad	Simulation et infrastructure ROS 2	<code>control_manager</code> , arbitrage automatique/manuel, rampes et sécurités, interface GUI PySide6, lancement et intégration des piles.
ZHANG Chengbo	Contrôle, perception et robustesse	<code>path_tracker</code> Pure Pursuit/Stanley/MPC, localisation SLAM, approche Pure Pursuit + LiDAR, injecteurs de latence et bruit d'odométrie.

Les tâches transverses ont été prises en charge collectivement : définition de l'architecture, réglage des paramètres, campagnes expérimentales, analyse des résultats, rédaction du rapport et préparation de la présentation.

L'objectif opérationnel est de boucler un circuit le plus rapidement possible sans collision ni sortie de piste. Les contraintes principales sont la latence perception-commande, le suivi précis d'une trajectoire de course et la stabilité à haute vitesse. Le projet vise donc non seulement un temps au tour faible, mais aussi une solution répétable, robuste aux perturbations et compatible avec les contraintes d'un véhicule à direction Ackermann.

2 Architecture du système

L'architecture suit une boucle fermée perception – décision – action. Gazebo publie l'odométrie et le LiDAR ; la localisation fusionne odométrie, TF et SLAM ; des injecteurs optionnels perturbent l'état ; les planificateurs produisent objectifs, trajectoire et vitesse cible ; le contrôleur calcule la commande ; le gestionnaire de contrôle arbitre avec la commande manuelle et renvoie `/autocar/cmd_vel` au simulateur. Les choix d'intégration s'appuient sur l'écosystème ROS 2/Gazebo et sur une séparation explicite des responsabilités entre nœuds [1], [2], [3], [4].

**Figure 1** – Architecture ROS 2 détaillée : simulation, localisation, injecteurs, planification, suivi de trajectoire, arbitrage et GUI.**Table 2** – Interfaces logicielles principales.

Couche	Nœuds / rôles	Topics principaux
Simulation	Gazebo 11, plugin Ackermann, LiDAR	<code>/autocar/cmd_vel</code> , <code>/autocar/odom</code> , <code>/scan</code>
Localisation	<code>slam_toolbox</code> + fusion odométrie/TF	<code>/autocar/state2D_raw</code> , <code>/autocar/state2D</code> , TF map->odom
Perturbations	<code>latency_injector</code> et <code>odom_noise_injector</code>	<code>latency_ms</code> , <code>odom_noise_std</code> , état perturbé
Planification	<code>global_planner</code> + <code>local_planner</code>	<code>/autocar/goals</code> , <code>/autocar/path</code> , <code>/autocar/target_velocity</code>
Contrôle	<code>path_tracker</code> + <code>control_manager</code>	<code>/autocar/auto_cmd_vel</code> , <code>/autocar/manuel_cmd_vel</code> , <code>/autocar/cmd_vel</code>

3 Méthodologie et implémentation

3.1 Sélection et réglage du contrôleur

Trois contrôleurs ont été évalués sur un circuit en anneau : Stanley, MPC et Pure Pursuit. Stanley corrige l'erreur latérale et l'erreur de cap [5], MPC anticipe l'évolution dynamique sous contraintes [6], et Pure Pursuit suit géométriquement un point de visée situé à une distance L_d [7]. Les résultats initiaux ont montré que Pure Pursuit était le plus performant et le plus simple à stabiliser pour cette plateforme.

La loi de braquage utilisée est donnée par :

$$\delta = \arctan\left(\frac{2L \sin \alpha}{L_d}\right), \quad (1)$$

où L est l'empattement, α l'angle vers le point de visée et L_d la distance de lookahead.

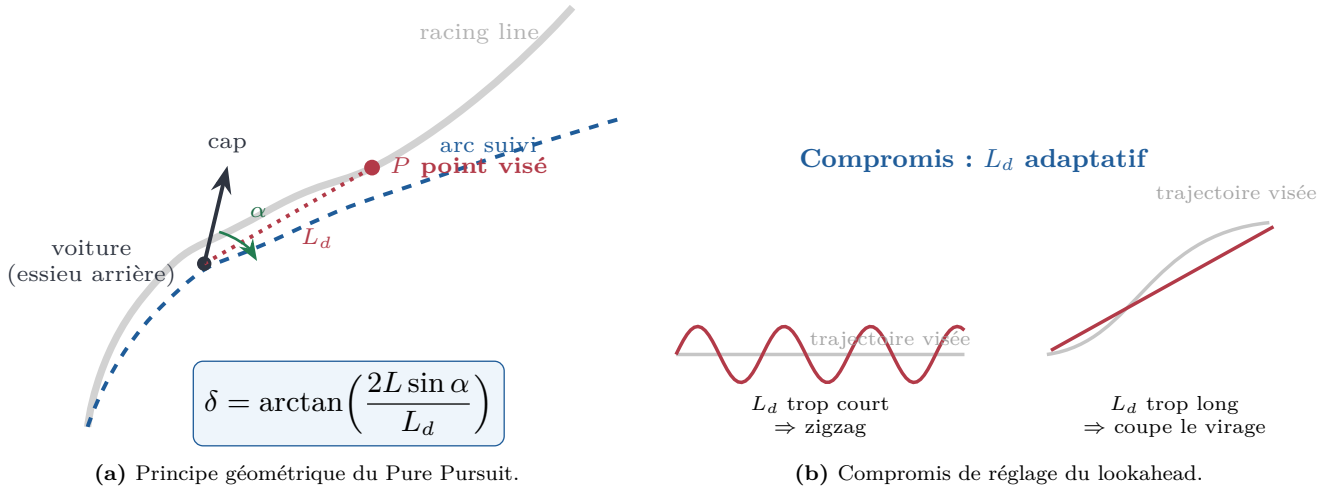


Figure 2 – Visualisation LaTeX/TikZ de la loi de suivi et de son compromis principal.

Un L_d trop court provoque du zigzag, alors qu'un L_d trop long coupe les virages. Le réglage retenu est adaptatif : L_d croît avec la vitesse et reçoit une extension uniquement en ligne droite pour réduire l'oscillation sans élargir les virages. Une écriture synthétique du réglage est :

$$L_d = \text{clip}(kv + L_{\text{extra}} \mathbb{1}_{|\kappa| < \kappa_s}, L_{\min}, L_{\max}), \quad (2)$$

où κ désigne la courbure locale et κ_s un seuil de ligne droite.

Table 3 – Paramètres décisifs du Pure Pursuit open-loop.

Paramètre	Valeur finale	Effet observé
lookahead_gain	0,4	Met L_d à l'échelle de la vitesse ; une valeur plus élevée rend la voiture plus douce mais peut couper les virages.
lookahead_min	1,5 m	Stabilise les basses vitesses et évite un comportement nerveux.
cruise_velocity	8,0 m/s	Fixe le rythme en ligne droite ; au-delà, la voiture sort de piste.
max_lateral_accel	5,5 m/s ²	Plafonne la vitesse en virage par $v \leq \sqrt{a_{\text{lat,max}}/\kappa}$.
steering_rate_limit	4,0 rad/s	Lisse le braquage et supprime les oscillations fortes.

3.2 Racing line et profil de vitesse

La ligne centrale n'est pas optimale pour un véhicule de course : elle allonge certains virages et augmente la courbure instantanée. Le projet génère donc une *racing line* hors ligne en minimisant la courbure sous contrainte de largeur de piste, puis applique un lissage. Le profil de vitesse limite la vitesse en virage avec :

$$v_{\text{target}} = \min\left(v_{\text{cruise}}, \sqrt{\frac{a_{\text{lat,max}}}{\max(\kappa, \kappa_{\min})}}\right), \quad (3)$$

et laisse le véhicule atteindre la vitesse de croisière en ligne droite.

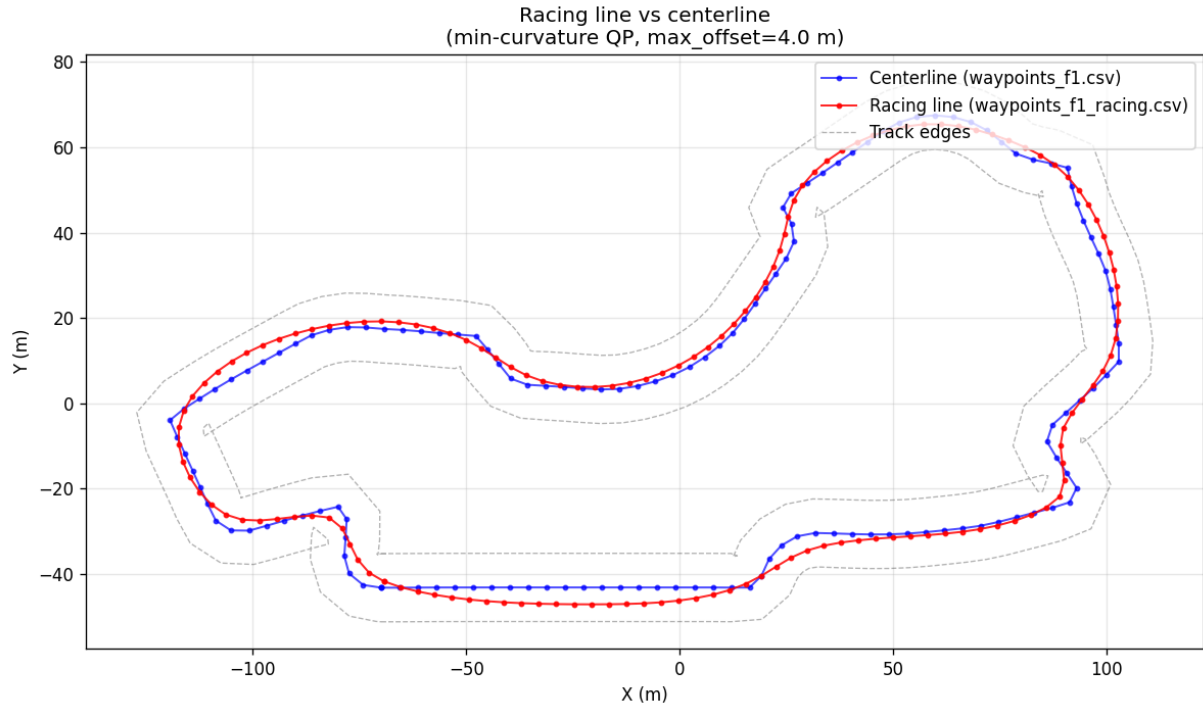


Figure 3 – Comparaison entre la ligne centrale et la *racing line* optimisée sur le circuit F1. La trajectoire rouge résulte d’une optimisation à courbure minimale sous contrainte de largeur de piste ($\text{max_offset} = 4,0 \text{ m}$), ce qui permet de lisser les virages et de réduire la courbure locale par rapport à la ligne centrale.

3.3 Pile Pure Pursuit + LiDAR + SLAM

Une seconde pile vise l’autonomie sans waypoints pré-calculés. Au premier tour, le véhicule explore avec le LiDAR et un comportement Follow-the-Gap, construit la carte SLAM et enregistre sa trajectoire. Aux tours suivants, le planificateur dérive une trajectoire fermée, optimise une *racing line* à partir de ce qui a été découvert, la lisse et la suit avec Pure Pursuit. Cette approche est plus sûre et plus générale, mais plus lente et plus sensible aux erreurs de localisation.

4 Résultats expérimentaux

4.1 Protocole

Les campagnes de test excluent un tour de chauffe lorsque c’est pertinent. Les métriques principales sont le temps au tour, la vitesse moyenne, l’erreur latérale RMS, le pic de taux de braquage et le nombre de sorties de piste. Un essai interrompu avant la fin est noté *timeout*. Les visualisations suivantes ont été reconstruites directement en PGFPlots/TikZ afin de conserver des graphiques vectoriels dans le rapport.

4.2 Stage I : sélection du contrôleur sur circuit en anneau

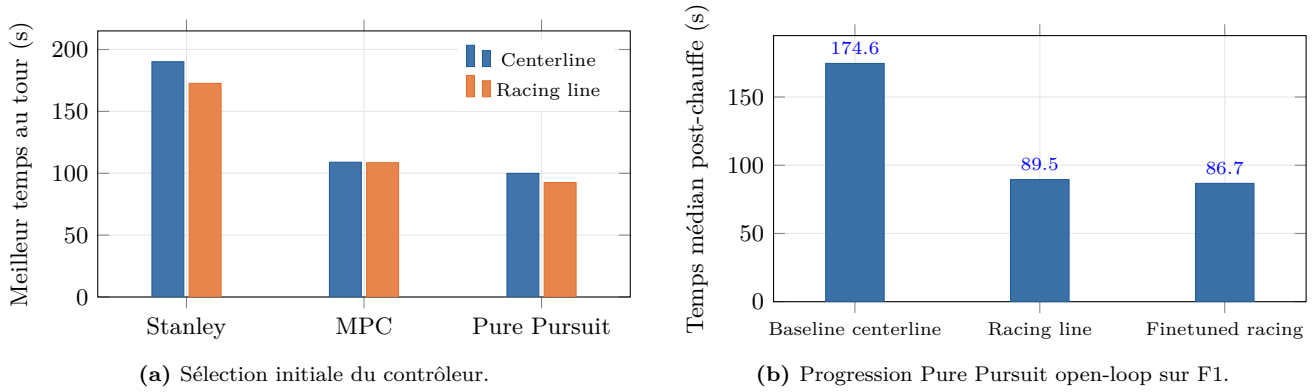


Figure 4 – Comparaison initiale des contrôleurs et progression de la pile Pure Pursuit sur F1.

Table 4 – Stage I : sélection de contrôleur sur circuit en anneau.

Contrôleur	Ligne	$T_{\min}$ (s)	T_{moy} (s)	$\bar{v}$ (m/s)	Gain racing
Stanley	centerline	190,1	191,1	3,42	–
Stanley	racing	172,6	173,1	3,63	-9,4 %
MPC	centerline	108,9	111,1	5,89	–
MPC	racing	108,5	110,4	5,71	-0,6 %
Pure Pursuit	centerline	99,9	105,4	6,23	–
Pure Pursuit	racing	92,5	95,0	6,63	-9,8 %

Pure Pursuit obtient le meilleur tour (92,5s) et le plus grand gain relatif apporté par la *racing line*. MPC reste compétitif, mais ne bénéficie presque pas de la *racing line* dans ce réglage ; Stanley est nettement plus lent.

4.3 Stage II : F1 open-loop, ligne centrale, racing line et tuning

Table 5 – Stage II : Pure Pursuit sur le circuit F1.

Profil	Ligne	T_{med} (s)	$\bar{v}$ (m/s)	ε_{RMS} (m)	$\dot{\delta}_{\text{max}}$ (rad/s)
baseline	centerline	174,6	3,52	0,768	12,46
racing	racing	89,5	6,41	0,295	5,07
finetuned	racing	86,7	6,61	0,351	1,33
finetuned perturbé	racing	98,7	18,55*	0,619	3,00

* Vitesse moyenne non comparable dans le run perturbé, car la localisation gonfle la distance parcourue.

Le passage centerline $\rightarrow$ *racing line* divise presque par deux le temps au tour (174,6s $\rightarrow$ 89,5s) et réduit fortement l'erreur RMS. Le réglage fin apporte un gain plus modeste en chrono (86,7s), mais transforme la stabilité : le pic de taux de braquage passe de 5,07 rad/s à 1,33 rad/s. Le profil agressif, avec 9–10 m/s ou $a_{\text{lat,max}} = 6,5 \text{ m/s}^2$, provoque des sorties de piste sans améliorer le chrono.

4.4 Stage III : pile Pure Pursuit + LiDAR + SLAM

Table 6 – Stage III : Pure Pursuit + LiDAR + SLAM en conditions nominales.

Profil	T_{med} (s)	$\bar{v}$ (m/s)	v_{max} (m/s)	Distance (m)	ε_{RMS} (m)	$\dot{\delta}_{\text{max}}$	Sorties
baseline	125,6	4,17	5,77	523,4	0,071	0,80	0
finetuned	115,1	4,71	7,25	542,1	0,108	0,60	0

La pile autonome boucle un tour complet sans carte préalable en 115,1 s après tuning, avec zéro sortie de piste. Elle reste environ 28 s plus lente que l'open-loop à waypoints, ce qui correspond au coût de l'autonomie et de la prudence imposée par la perception en ligne.

5 Robustesse : latence et bruit odométrique

Le bonus du projet a été traité par un balayage croisé de la latence injectée $L \in \{0, 200, 500\}$ ms et du bruit odométrique $\sigma \in \{0, 0,05, 0,10\}$ m. Le but est de mesurer non seulement le temps au tour, mais aussi la stabilité et la dégradation de trajectoire quand l'état véhicule devient retardé ou bruité.

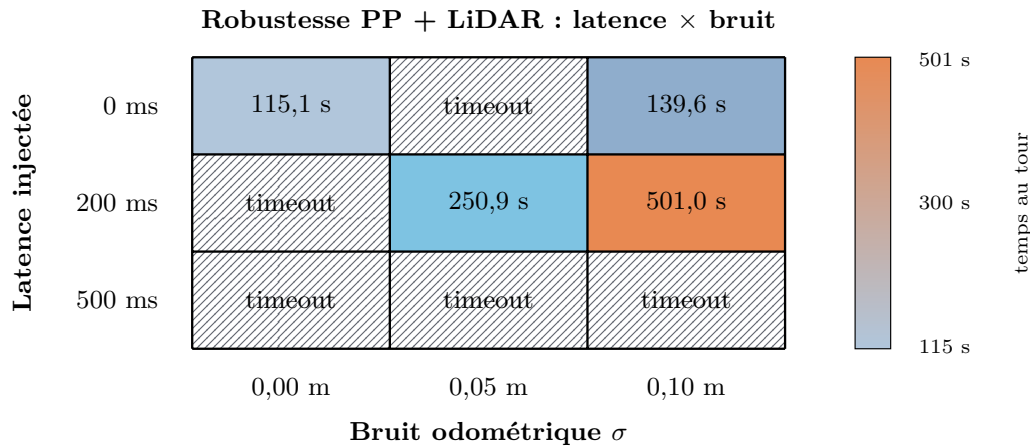


Figure 5 – Dégradation des performances du stack Pure Pursuit + LiDAR sous latence et bruit odométrique. Les cases hachurées correspondent à des *timeouts*.

Table 7 – Matrice de robustesse latence $\times$ bruit.

Latence / bruit	$\sigma = 0,00$ m	$\sigma = 0,05$ m	$\sigma = 0,10$ m	Interprétation
0 ms	115,1 s	timeout	139,6 s	Le bruit seul peut suffire à rendre la localisation limite ; le comportement est non monotone.
200 ms	timeout	250,9 s	501,0 s	La performance s'effondre ; la sécurité reste globalement préservée.
500 ms	timeout	timeout	timeout	La latence dépasse l'enveloppe opérationnelle du stack.

La limite pratique de la pile Pure Pursuit + SLAM se situe sous environ 200 ms de latence. La combinaison latence + bruit augmente la distance réellement parcourue et le temps au tour, même lorsque la voiture ne sort pas de piste. Cela montre que le système est davantage limité par la qualité temporelle de la localisation que par le contrôleur géométrique lui-même.

6 Discussion : conservateur vs agressif

Table 8 – Comparaison demandée entre réglage conservateur et agressif.

Réglage	Paramètres typiques	Résultat	Conclusion
Conservateur	$cruise = 8,0$ m/s ; $a_{lat,max} = 5,5$ m/s ² ; $steering_rate_limit = 4,0$ rad/s	86,7 s, zéro sortie de piste, suivi répétable	Meilleur compromis pour ce contrôleur.
Agressif	$cruise = 9-10$ m/s ou $a_{lat,max} = 6,5$ m/s ²	crash ou sortie de piste, aucun gain validé	Contre-productif avec Pure Pursuit géométrique.

L'expérience confirme que la vitesse brute ne suffit pas : le gain principal vient de la géométrie de trajectoire. Une trajectoire à courbure réduite permet d'augmenter la vitesse moyenne sans dépasser la limite latérale, alors qu'un simple relèvement de consigne met le véhicule en survirage/sous-virage et dégrade la sécurité.

7 Limites et validité des essais

Les résultats sont obtenus en simulation, sur un véhicule Ackermann et des circuits mis à l'échelle. Les conclusions sont donc solides pour comparer les piles entre elles, mais elles ne remplacent pas une validation sur plateforme réelle avec pneus, adhérence variable, saturation moteur et délais réseau mesurés. La vitesse moyenne du run perturbé open-loop n'est pas utilisée pour comparer les performances, car la distance estimée diverge sous bruit de localisation.

Table 9 – Limites expérimentales et mesures de contrôle.

Point de vigilance	Impact potentiel	Mesure prise
Simulation Gazebo	Dynamique plus simple qu'un véhicule réel.	Comparaisons réalisées à protocole identique.
Waypoints pré-calculés	Performance élevée mais dépendante du circuit connu.	Pile Pure Pursuit + LiDAR + SLAM testée sans carte préalable.
Localisation bruitée	Temps au tour et distance peuvent devenir non comparables.	Matrice latence $\times$ bruit et analyse qualitative de stabilité.

8 Leçons apprises

Les expérimentations ont montré que la géométrie de la trajectoire est plus déterminante que la vitesse brute pour améliorer les performances. En effet, l'optimisation de la *racing line* constitue le principal levier de gain de temps au tour, tandis qu'une simple augmentation de la vitesse de consigne tend à provoquer davantage de sorties de piste.

Par ailleurs, le lookahead adaptatif s'est révélé être un élément clé pour la stabilité du véhicule : son extension dans les lignes droites réduit significativement les oscillations sans dégrader la capacité à négocier les virages. La comparaison précoce des différents contrôleurs a également permis d'éviter des phases de réglage inutiles ; le circuit en anneau a notamment servi à écarter rapidement l'approche Stanley et à mieux situer les performances du MPC avant l'évaluation du contrôleur F1.

Les résultats mettent aussi en évidence un compromis entre autonomie et performance : l'architecture combinant Pure Pursuit, LiDAR et SLAM offre une solution plus générale et plus sûre, mais reste environ 28 s plus lente que l'approche en boucle ouverte. Enfin, les essais ont confirmé l'importance de tester la robustesse dès les premières phases de développement, les injections de latence et de bruit permettant de révéler des limitations qui demeurent invisibles dans des conditions idéales.

9 Conclusion et perspectives

Le projet livre un nœud de contrôle autonome haute vitesse basé sur Pure Pursuit, intégré à une architecture ROS 2 complète avec simulation, localisation, planification, gestion de commande et analyse de robustesse. Le meilleur résultat open-loop atteint 86,7 s par tour sur le circuit F1 mis à l'échelle, sans sortie de piste. La version Pure Pursuit + LiDAR + SLAM ajoute la découverte du circuit à la volée et atteint 115,1 s par tour, également sans sortie nominale.

Les perspectives principales sont les suivantes :

- passer à un contrôleur dynamique, MPC ou Vector Pursuit, pour dépasser le plafond d'environ 8 m/s ;
- améliorer la fusion et le filtrage de localisation pour mieux supporter la latence ;
- packager le contrôleur sous forme de plugin Nav2 Regulated Pure Pursuit lorsque l'interface logicielle sera stabilisée.

Bibliographie

- [1] OPEN ROBOTICS, *ROS 2 Humble Hawksbill Documentation*, Documentation officielle, 2026. adresse : <https://docs.ros.org/en/humble/index.html>.
- [2] OPEN ROBOTICS, *Gazebo Classic Documentation*, Documentation officielle, 2026. adresse : <https://classic.gazebosim.org/>.
- [3] S. MACENSKI et CONTRIBUTORS, *slam_toolbox*, Dépôt GitHub, 2026. adresse : https://github.com/SteveMacenski/slam_toolbox.
- [4] OPEN NAVIGATION LLC, *Nav2 Documentation*, Documentation officielle, 2026. adresse : <https://navigation.ros.org/>.

- [5] S. THRUN et al., « Stanley : The Robot that Won the DARPA Grand Challenge, » *Journal of Field Robotics*, t. 23, n° 9, p. 661-692, 2006. DOI : [10.1002/rob.20147](https://doi.org/10.1002/rob.20147). adresse : <https://robots.stanford.edu/papers/thrun.stanley05.pdf>.
- [6] J. B. RAWLINGS, D. Q. MAYNE et M. M. DIEHL, *Model Predictive Control : Theory, Computation, and Design*, 2^e éd. Nob Hill Publishing, 2017. adresse : <https://sites.engineering.ucsb.edu/~jbraw/mpc/>.
- [7] R. C. COULTER, « Implementation of the Pure Pursuit Path Tracking Algorithm, » Carnegie Mellon University, Robotics Institute, rapp. tech., 1992. adresse : https://www.ri.cmu.edu/pub_files/pub4/coulter_r_craig_1992_1/coulter_r_craig_1992_1.pdf.